

实验 2：开环控制

概述

在本次实验中，我们将使用**开环控制**策略（即只发送指令，不使用反馈）移动 Turtlebot 机器人。这个过程中大家可以熟悉一下机器人的一些基本操作，并尝试调试参数。

本次实验的任务是让机器人以正方形轨迹移动，沿途访问四个导航点 $[4, 0]$ 、 $[4, 4]$ 、 $[0, 4]$ 和 $[0, 0]$ 。具体来说，需要控制机器人向前移动 4 米，左转 90 度，再向前移动 4 米，以此类推，直到回到 $[0, 0]$ 点。请注意，机器人在完成这个正方形运动后，应该回到 $[0, 0]$ 点附近。

本次实验的位置控制精度要求为 1 米，也就是说最终回到距离 $[0, 0]$ 点的 1 米范围之内都可以。（在下一章节的实验中，我们会对小车进行闭环控制，届时会对位置控制有更高的精度要求。）

为了更好地照顾初学者，我们提供了 Python 的示例代码和分步说明。不熟悉 ROS 的同学也不妨利用这个机会进行学习。如果熟悉机器人开发，可以使用自己的程序代码进行控制，只要能够完成任务即可。

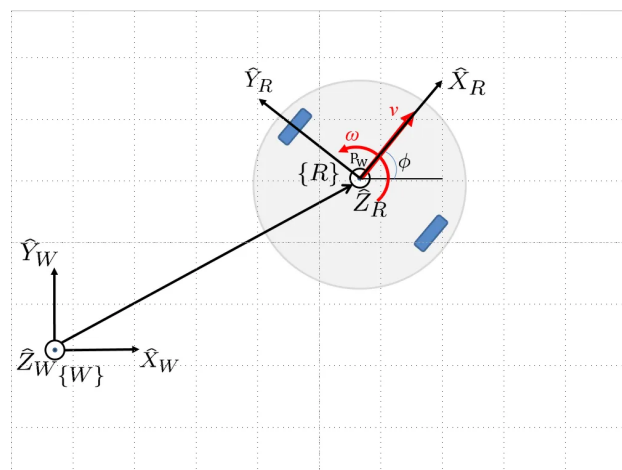
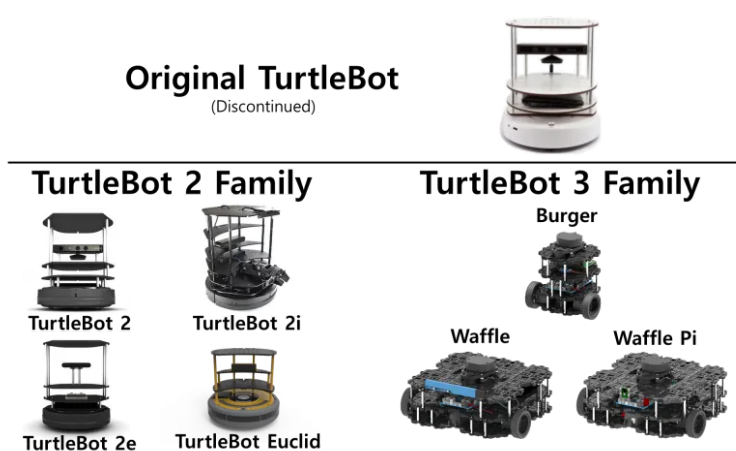
作业提交

- 需要提交的文件
 - `open_loop.py`
 - 以 PDF 文件格式提交实验报告，包含方形的轨迹图和机器人在 Gazebo 仿真器中到达导航点的截图
- 评分标准：
 - 机器人能够访问正方形轨迹的所有四个导航点（误差小于 1.0 米）。将根据访问导航点数量给予部分成绩。

Turtlebot 简介

Turtlebot 系列的机器人是差速轮式机器人。有关 Turtlebot 机器人家族的具体情况，请参见下图（左）。

即便机器人的主体部分有许多传感器和支架结构，但这种机器人的运动学模型是可以根据其底座的移动特点进行提炼和简化的。由于底座是使用两个轮子，一左一右围绕其中心对称分布，从而差速驱动的，所以这个机器人的运动学模型是**差速轮模型**（Differential Drive）。参见下图（右）。



作为用户，我们可以向机器人发送上层运动控制指令（线速度 v 和角速度 ω ）。移动底座首先会将相对于机器人中心的 v 和 w 的命令转换为每个轮子所需的旋转速度，然后再通过反馈控制，调整驱动轮的电机电流，来控制最终的旋转速度。

为了描述机器人的位置和方向，我们为其附加了一个机器人坐标系 $\{R\}$ 。该坐标系的原点位于其动力轮之间的中心位置；该坐标系的 X 轴指向前方（沿线速度 v 的方向），Y 轴指向左侧，Z 轴指向上方。

为了跟踪机器人的位置和方向，我们通常会在机器人移动的同一平面内定义一个世界坐标系 $\{W\}$ 。此外，机器人和世界坐标系之间的任何旋转都可以表示为围绕 Z 轴的旋转。因此，机器人相对于世界坐标系的位置为

$$P_W = \begin{bmatrix} x \\ y \\ 0 \end{bmatrix}$$

而用于表示机器人坐标系 $\{R\}$ 相对于世界坐标系 $\{W\}$ 的方位/朝向的旋转矩阵则为

$$R_{WR} = \begin{bmatrix} \cos(\phi) & -\sin(\phi) & 0 \\ \sin(\phi) & \cos(\phi) & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

编程提示

1. 我们按照 ROS 的惯例使用[国际单位制](#)。（例如，长度以米为单位，时间以秒为单位，角度以弧度为单位）。更多信息，请参阅 ROS Wiki 文章[REP 103 标准计量单位和坐标约定](#)。
2. 在机器人开机时，会根据当时的朝向确定一个里程计（odom）坐标系。这个坐标系默认当前的前进方向为 x 轴正方向，左转方向为 y 轴正方向，再根据右手定则可得 z 轴向上。这在[REP 103](#)中也有规定。机器人其后所得到的轮式里程计的位置反馈，都是相对于这个 odom 坐标系而言的，并非真正的绝对的地图/世界坐标系。

3. 与 C/C++ 不同，Python 编程语言对缩进很敏感。
 - 你可以使用 `space` 或 `Tab` 来缩进，但请不要在一个文件中混合使用。两者只能选其一，否则，你会看到语法错误。（我个人建议使用 `space`）。
 - 更好的方法是使用集成开发环境（IDE）进行编程，`Tab` 键可以自动转换为 4 个空格。
4. 在 Linux 下，推荐使用的集成开发环境是 `VS Code`。只需登录官方网站，下载 `.deb` 文件并安装即可。
5. 在 Gazebo 仿真器中，你可以使用 `Ctrl + R` 快捷键将机器人设置回原点，而无需重新启动仿真器。
6. 在本实验中，你需要微调开环控制参数。
 - 请注意，不同平台的参数可能有所不同。换句话说，在你的虚拟机上运行效果好的参数不一定能在其他机器上复现出同样的运行轨迹。
 - 在 Gazebo 中，地面网格的大小为 1 米，你可以参考网格来估算机器人走了多远，从而调整开环控制参数。

示例代码

示例代码可让机器人向前移动一定距离。你需要在 `run` 函数下进行修改，以完成方形轨迹。

- 将我们提供的示例程序 `open_loop.py` 文件移动到桌面，然后运行以下命令：

代码块

```
1 cd ~/ros2_ws/src/robotics101
2 mkdir scripts
3 cd scripts
4 mv ~/Desktop/open_loop.py ~/ros2_ws/src/robotics101/scripts
```

- 回到终端，有两种运行方式。一种是将此脚本作为 Linux 下 Python 程序的输入，如下所示。

代码块

```
1 python3 open_loop.py
```

- 另一种方法是将其作为 Linux 中的常规可执行文件运行。在这种情况下，首先需要授予该 Python 脚本执行权限。这一步只需运行一次。

代码块

```
1 chmod +x open_loop.py
```

- 现在你可以看到，当你在终端中 `ls` 当前目录时，这个文件是绿色的。这就是 Linux 终端区分可执行文件（绿色或高亮）和不可执行文件（白色）的方法。
- 然后你就可以用以下命令来运行

代码块

```
1 ./open_loop.py
```

- 现在请编辑 `open_loop.py` 文件，使其能够访问我们之前提到的四个导航点。你可以自己定义前进、转弯等功能的函数，然后灵活地调用他们，来完成一个完整的方形轨迹。
- 程序编写好了之后，就可以用和之前同样的方法来启动 Turtlebot 机器人，然后再在另一个终端中运行 Python 程序进行测试。

代码块

```
1 ros2 launch turtlebot3_gazebo empty_world.launch.py
```